

Senaryo ID	UC01 – Kayıt Olma (Register)
Senaryo Adı	Yeni Kullanıcı Kaydı
Ana Aktör	Ziyaretçi (Misafir Kullanıcı)
Ön Koşullar	Ziyaretçinin girdiği e-posta adresi veritabanında (MongoDB) daha önce kayıtlı olmamalıdır.
Hedef Şartlar	- Ad, soyad, e-posta ve şifre eksiksiz alınmalıdır. - İşlem başarılı olduğunda veritabanında yeni bir user dokümanı oluşmalı ve HTTP 201 dönmelidir.
Ana Başarısı Senaryosu	
- Ziyaretçi, "Kayıt Ol" sayfasına gider ve formu doldurur. - Ziyaretçi "Kayıt Ol" butonuna basar. - Sistem, e-posta adresinin kullanımda olup olmadığını kontrol eder. - E-posta boşta ise ve şifre formatı uygunsa, sistem verileri kaydeder. - Kullanıcıya başarı mesajı ("Kullanıcı başarıyla oluşturuldu") gösterilir ve Giriş sayfasına yönlendirilir.	
Genişlemeler	
3a: E-posta zaten sistemde kayıtlıysa: Sistem (HTTP 409), "Bu email adresi zaten kullanımda." uyarısı verir. Kayıt işlemi iptal edilir. 4a: Zorunlu alanlar (Ad/Soyad/Şifre) boşsa: Sistem (HTTP 400), "Ad, soyad, e-posta ve şifre zorunludur" hatası döndürür.	

Senaryo ID	UC02 – Giriş Yapma (Login)
Senaryo Adı	Kullanıcı Girişi ve Yetkilendirme
Ana Aktör	Kayıtlı kullanıcı
Ön Koşullar	Kullanıcının sistemde geçerli bir hesabı bulunmalıdır.
Hedef Şartlar	- E-posta ve şifre eşleştğinde sistem bir JWT Token veya oturum yetkisi vermelidir. - Güvenlik gereği, API yanıtında kullanıcının şifresi geri dönülmemelidir.
Ana Başarısı Senaryosu	
- Kullanıcı "Giriş" sayfasına gider. - E-posta ve şifresini girip "Giriş Yap" butonuna tıklar. - Sistem veritabanında bu bilgileri eşleştirir. - Eşleşme başarılıysa (HTTP 200), sistem bir oturum anahtarı (Token) oluşturur ve kullanıcı profil bilgileriyle beraber Frontend'e iletir. - Kullanıcı, yetki gerektiren sayfalara (Örn: İçerik Ekleme, Profil) erişim kazanır.	
Genişlemeler	
3a: E-Posta veya şifre hatalıysa: Sistem (HTTP 401), "Hatalı email veya şifre" uyarısı verir ve token üretmez.	

Senaryo ID	UC03 – Profil Görüntüleme (View Profile)
Senaryo Adı	Kullanıcı Profili ve Yazılarını Görüntüleme
Ana Aktör	Kayıtlı Kullanıcı
Ön Koşullar	Görüntülenmek istenen kullanıcının sistemde geçerli bir hesabı (ID) bulunmalıdır.
Hedef Şartlar	- İlgili kişiye ait genel profil bilgileri (bio, meslek, fotoğraf vb.) ile birlikte oluşturduğu tüm blog yazıları getirilmelidir. - Güvenlik gereği, API yanıtından kullanıcının şifresi kesinlikle temizlenmiş olmalıdır.
Ana Başarısı Senaryosu	
- Aktör, platformda dolaşırken bir blog kartındaki yazar adına/fotoğrafına tıklar veya doğrudan bir profil URL'sine (Örn: /profile/12345) gider. - Sistem (Frontend), ilgili yazarın ID'sini alarak arka plana GET /api/users/{id} isteği atar. - Backend, User koleksiyonundan o ID'ye sahip kullanıcıyı bulur ve userToPublic fonksiyonundan geçirerek şifre (password) bilgisini gizler. - Backend, eşzamanlı olarak Post koleksiyonunda authorId alanı bu kullanıcının ID'sine eşit olan tüm blog yazılarını sorgular. - Bilgiler başarıyla toplanırsa (HTTP 200), kullanıcı bilgileri ve makaleleri JSON formatında tek bir paket olarak arayüze iletilir. - Frontend, sayfanın üst kısmında kullanıcı profil kartını, alt kısmında ise yazarın makalelerini (grid/liste halinde) ekrana basar.	
Genişlemeler	
3a: İstenen kullanıcı ID'si sistemde kayıtlı değilse veya hesap silinmişse: Sistem (HTTP 404), "Kullanıcı bulunamadı." uyarısı verir. Kullanıcıya arayüzde "Böyle bir profil bulunmamaktadır" veya "404 Not Found" ekranı gösterilir. 3b: Gönderilen ID, standart MongoDB formatına (ObjectId) uygun değilse: Başarısızlık: Sistem (HTTP 500), "Geçersiz ID formatı" hatası fırlatır.	

Senaryo ID	UC04 – Profil Güncelleme (Update Profile)
Senaryo Adı	Profil Bilgilerini Düzenleme
Ana Aktör	Kayıtlı Kullanıcı
Ön Koşullar	Kullanıcı sisteme giriş yapmış olmalı (geçerli bir JWT token'a sahip olmalı) ve güncellediği profilin ID'si, kendi oturum ID'si ile eşleşmelidir.
Hedef Şartlar	- Yalnızca değiştirilen (kirli/dirty) veriler backend'e gönderilmeli ve güncellenmelidir. - E-posta ve şifre gibi hassas bilgilerin değiştirilmesi bu uç nokta üzerinden yapılmamalıdır (ayrı bir kimlik doğrulama gerektirir). - İşlem tamamlandığında güncel profil verisi HTTP 200 ile dönmelidir.
Ana Başarısı Senaryosu	
- Kullanıcı formdaki bir veya birkaç alanı değiştirir (Örn: "Bio" metnini günceller) ve "Değişiklikleri Kaydet" butonuna tıklar. - Sistem (Frontend), değiştirilen verileri bir JSON nesnesi (patch objesi) olarak hazırlayıp PUT /api/users/{id} uç noktasına gönderir. - Backend, güncellenmek istenen kullanıcının sistemde var olduğunu (findById) doğrular. - Backend, gelen yeni verileri (username, bio, profession, vb.) mevcut kullanıcı verilerinin üzerine yazar (findByIdAndUpdate). - Sistem, güncellenmiş kullanıcı bilgilerini (şifre hariç userToPublic ile temizlenmiş şekilde) JSON olarak döner (HTTP 200). - Ekranda "Profil başarıyla güncellendi" (toast) mesajı gösterilir ve kullanıcı profiline yönlendirilir.	
Genişlemeler	
5a: Kullanıcı ID'si sistemde bulunamazsa (veya geçersizse): Sistem (HTTP 404), "Kullanıcı bulunamadı." uyarısı verir ve işlemi iptal eder. 4a: Yetkisiz bir kullanıcı başka birinin profilini düzenlemeye çalışırsa (Backend Guard): Sistem (HTTP 403 Forbidden), "Bu profili düzenleme yetkiniz yok" uyarısı verir. 6a: Sunucu veya veritabanı hatası oluşursa: Sistem (HTTP 500) hata fırlatır ve "Güncelleme sırasında bir hata oluştu" mesajı gösterilir.	

Senaryo ID	UC05 – Hesap Silme (Delete Account)
Senaryo Adı	Kullanıcı Hesabını ve Bağlı İçerikleri Kalıcı Olarak Silme
Ana Aktör	Kayıtlı Kullanıcı
Ön Koşullar	• Kullanıcı sisteme giriş yapmış olmalıdır (Geçerli bir Token bulunmalıdır). • Silinmek istenen hesabın ID'si ile istekte bulunan kullanıcının ID'si eşleşmelidir.
Hedef Şartlar	• Yıkıcı (Destructive) bir işlem olduğu için işlem öncesi kesinlikle kullanıcı onayı alınmalıdır. • Kullanıcının hesabı (User), tüm blog yazıları (Post) ve bu yazılara bağlı olan/kullanıcının kendi yazdığı tüm yorumlar (Comment) kalıcı olarak temizlenmelidir. • İşlem sonrası kullanıcı sistemden çıkış (Logout) yapılmış duruma getirilmelidir.
Ana Başarısı Senaryosu	
1. Kullanıcı, hesap yönetimi panelinden "Hesabımı Sil" butonuna tıklar ve çıkan güvenlik uyarısını (Modal) onaylar. 2. Sistem (Frontend), DELETE /api/users/{id} isteğini kullanıcının oturum ID'si ile birlikte sunucuya gönderir. 3. Backend, "zincirleme silme" (Cascade Delete) mantığıyla önce kullanıcının yazdığı tüm makaleleri ve bağlantılı/kendi yazdığı yorumları veritabanından temizler. 4. Ardından kullanıcının ana profil kaydı (User dokümanı) silinir ve işlem başarılı (HTTP 204) olarak sonuçlandırılır. 5. Sistem, cihazdaki oturum anahtarını (Token) temizler, ekranda başarı mesajı gösterir ve kullanıcıyı Ana Sayfaya yönlendirir.	
Genişlemeler	
• 1a: Kullanıcı onayı iptal ederse: Sistem işlem yapmaz, uyarı penceresi kapanır. • 2a: Yetkisiz bir işlem denenirse: Sistem (HTTP 403), oturum açmış kişi ile silinmek istenen hesap eşleşmiyorsa işlemi reddeder. • 3a: Veritabanı hatası oluşursa: Sistem (HTTP 500) hata fırlatır ve "İşlem sırasında bir hata oluştu" mesajı gösterilir.	

Senaryo ID	UC06 – Blog Yazısı Oluřturma (Create a Blog Post)
Senaryo Adı	Blog Yazısı Oluřturma
Ana Aktör	Kayıtlı Kullanıcı (Yazar)
Ön Kořullar	- Kullanıcı siteye kayıt olmuř ve e-posta/řifre ile giriř yapmıř olmalıdır (Geçerli bir JWT Bearer token’a sahip olmalı). - Uygulamanın veritabanı (MongoDB) baęlantısı aktif olmalıdır.
Hedef řartlar	- Kullanıcı; bařlık, ierik, kategori ve etiket bilgilerini eksiksiz girmelidir. - İřlem bařarılı olduęunda yazı sisteme kaydedilir ve genel blog listesinde (Ana Sayfa) görünür hale gelir. - Yazı oluřturulduktan sonra kullanıcı ilgili yazının detay sayfasına yönlendirilir.
Ana Bařarısı Senaryosu	
- Kullanıcı, zorunlu alanlar olan "Bařlık" ve "İerik" metinlerini doldurur. İsteęe baęlı olarak "Kategori" seer ve "Etiketler" (tags) ekler. - Kullanıcı, formu tamamladıktan sonra "Yayınla" (veya "Kaydet") butonuna basar. - Sistem (Backend /api/posts), kullanıcının yetkisini doęrular ve girilen bilgilerin eksiksiz olduęunu kontrol eder. - Bilgiler doęruysa, sistem verileri MongoDB veritabanına kaydeder (HTTP 201 Created). - Ekranda "Yazı bařarıyla oluřturuldu" mesajı (toast notification) gösterilir. - Sistem, kullanıcıyı yeni oluřturulan blog yazısının detay sayfasına (veya ana sayfaya) yönlendirir.	
Geniřlemeler	
5a: Kullanıcı bařlık veya ierik alanını boř bırakırsa: Sistem (Frontend veya Backend), "Bařlık ve ierik zorunludur" uyarısı verir. Form gönderilmez ve kullanıcıdan eksik alanları doldurması istenir. 5b: Kullanıcının oturum süresi (Token) dolmuřsa veya yetkisiz istek atılmıřsa: Sistem (HTTP 401 veya 403), "Oturum süreniz doldu, lütfen tekrar giriř yapın" mesajı verir. Kullanıcı doęrudan "Giriř Yap" sayfasına yönlendirilir ve yazı kaydedilmez. 6a: Veritabanı veya sunucu kaynaklı bir hata oluřursa: Sistem (HTTP 500), "Yazı oluřturulurken bir sunucu hatası oluřtu, lütfen daha sonra tekrar deneyin" uyarı mesajı verir ve hata kaydı (log) oluřturur.	

Senaryo ID	UC07 – Blog Yazısı Silme (Delete a Post)
Senaryo Adı	İçerik Kaldırma
Ana Aktör	Yazının Sahibi (Yazar)
Ön Koşullar	• Kullanıcı oturum açmış olmalı ve silinmek istenen yazının authorId değeri ile kullanıcının ID'si eşleşmelidir.
Hedef Şartlar	• Yazı silindiğinde, o yazıya ait tüm alt yorumlar (Comment koleksiyonu) da "Cascade Delete" mantığıyla veritabanından kalıcı olarak silinmelidir.
Ana Başarısı Senaryosu	
1. Kullanıcı, daha önce yazdığı bir yazının detay veya düzenleme sayfasına girer. 2. Ekranda bulunan kırmızı "Sil" butonuna tıklar. 3. Sistem güvenlik amaçlı bir "Emin misiniz?" pop-up onayı çıkarır. 4. Kullanıcı onaylar. Sistem, DELETE /api/posts/{id} isteği atar. 5. Sistem önce yazıya bağlı tüm yorumları (deleteMany), ardından blog yazısının kendisini (findByIdAndDelete) siler. 6. İşlem tamamlanır (HTTP 204 No Content), kullanıcı ana sayfaya yönlendirilir.	
Genişlemeler	
• 4a: Backend'de yetki eşleşmezse: Sistem (HTTP 403), "Bu yazıyı silme yetkiniz yok" uyarısı verir ve işlemi durdurur.	

Senaryo ID	UC08 – Blog Yazısı Listeleme (List a Blog Posts)
Senaryo Adı	Ana Sayfa Akışı ve Gelişmiş Arama
Ana Aktör	Ziyaretçi / Kullanıcı (Herkese Açık)
Ön Koşullar	Veritabanında (Post koleksiyonu) en az bir adet onaylı blog yazısı bulunmalıdır.
Hedef Şartlar	- Yazılar; sıralama (recent/popular), kategori (topic) veya arama sorgusuna (q) göre dinamik olarak filtrelenebilmelidir. - Yazıların beğeni ve yorum sayıları kart üzerinde görünmelidir.
Ana Başarısı Senaryosu	
- Aktör, sitenin "Ana Sayfasına" veya "Keşfet" sekmesine girer. - Sistem, varsayılan olarak yazıları "en yeniler" (sort=recent) düzeninde API'den çeker. - Aktör, arama çubuğuna bir kelime yazar (Örn: "Yapay zekâ") veya bir kategori seçer. - Sistem, aranan kelimeyi başlık, içerik ve etiketlerde (tags) Regex ile tarar. - Filtrelenen yazılar listelenir.	
Genişlemeler	
4a: Aranan kelimeye ait sonuç bulunamazsa: Sistem boş bir dizi ([]) döner ve ekranda "Sonuç bulunamadı" (Empty State) tasarımı gösterilir.	

Senaryo ID	UC09 – Yorum Ekleme (Add a Comment)
Senaryo Adı	Kullanıcı Etkileşimi
Ana Aktör	Kayıtlı Kullanıcı
Ön Koşullar	• Yorum yapılmak istenen Post (Yazı) veritabanında mevcut olmalıdır.
Hedef Şartlar	• Yorum metni boş olamaz. • Yorum başarılı şekilde eklendikten sonra anında sayfada listelenmelidir.
Ana Başarısı Senaryosu	
1. Kullanıcı, bir blog makalesinin okuma sayfasında en alta iner. 2. "Yorumunuzu yazın" metin kutusuna düşüncelerini girer ve "Gönder" butonuna basar. 3. Sistem metnin boş olup olmadığını kontrol eder. 4. Metin geçerliyse, postId, authorId ve content veritabanına kaydedilir (HTTP 201). 5. Yorum, sayfa yenilenmeden yazarın ismi ve tarihiyle birlikte ekranda gösterilir.	
Genişlemeler	
• 3a: Yorum alanı boşsa veya sadece boşluktan oluşuyorsa: Sistem (HTTP 400), "Yorum metni boş olamaz" hatası fırlatır ve işlemi reddeder.	

Senaryo ID	UC10 – Yorum Silme (Delete Comment)
Senaryo Adı	Kullanıcı Yorumunu Silme
Ana Aktör	Kayıtlı Kullanıcı (Yorum Sahibi)
Ön Koşullar	Kullanıcı sisteme giriş yapmış olmalı ve silinmek istenen yorum kullanıcının kendisine ait olmalıdır.
Hedef Şartlar	Yorum veritabanından kalıcı olarak silinmeli ve bağlı olduğu blog makalesinin altından anında kaybolmalıdır.
Ana Başarısı Senaryosu	
- Kullanıcı, bir blog makalesinin altında kendi yazdığı yorumun yanındaki "Sil" ikonuna (veya butonuna) tıklar. - Sistem (Frontend), yorumun ID'sini ve kullanıcının oturum ID'sini (authorId) alarak DELETE /api/comments/{id} isteğini sunucuya gönderir. - Backend, yorumun veritabanında mevcut olduğunu doğrular. - Backend, ilgili yorumu (Comment koleksiyonundan) kalıcı olarak siler ve HTTP 204 (No Content) başarılı durum kodunu döner. - Sistem (Frontend), sayfa yenilenmeden silinen yorumu ekrandan kaldırır ve "Yorum başarıyla silindi" bildirimi (toast) gösterir.	
Genişlemeler	
3a: Yorum veritabanında bulunamazsa (Örn: zaten silinmişse): Sistem (HTTP 404), "Yorum bulunamadı." uyarısı verir ve işlemi durdurur. 3b: Yetkisiz silme girişimi (Başkasına ait yorum): Backend güvenlik bariyeri, işlemi yapan kişi ile yorum sahibi eşleşmiyorsa isteği reddeder. (<i>Not: Mevcut kodunda yoruma alınmış olsa da mantık akışı bu şekilde olmalıdır</i>). 4a: Sunucu veya veritabanı hatası: Sistem (HTTP 500) hata fırlatır ve "İşlem sırasında bir hata oluştu" mesajı ekrana yansıtılır.	

Senaryo ID	UC11 – Yapay Zekâ ile Etkileşim
Senaryo Adı	AI Asistan ile Sohbet (Chatbot)
Ana Aktör	Sistem Kullanıcısı
Ön Koşullar	Kullanıcı bir mesaj yazmış olmalıdır. (Sunucu tarafında API_KEY aktif olmalıdır, aksi halde sistem "Akıllı Yedek Mod" ile cevap verir).
Hedef Şartlar	- Asistanın yanıtı teknoloji, yazılım ve platform odaklı kalmalıdır. - Sohbet geçmişi (history) taşınarak bağlam (context) korunmalıdır.
Ana Başarısı Senaryosu	
- Kullanıcı sitenin sağ alt köşesindeki veya menüdeki "AI Asistan" ikonuna tıklar. - Sohbet penceresi açılır, kullanıcı bir teknik soru veya tavsiye talebi yazar (Örn: "React'te state nasıl yönetilir?"). - Kullanıcı Enter veya Gönder tuşuna basar. - Sistem POST /api/chatbot endpoint'ine önceki konuşma geçmişi (history) ve yeni mesajı JSON olarak gönderir. - Backend, bu metni Google Gemini modeline (sistem talimatlarıyla birlikte) iletir. - Modelden dönen Markdown formatlı metin (HTTP 200) frontend'e ulaşır ve sohbet baloncuğu olarak ekrana basılır.	
Genişlemeler	
5a: API Key tanımlı değilse veya limit dolmuşsa: Sistem çökmez. Bunun yerine localAssistantReply devreye girer ve "Şu an yerel yedek moddasın, ... konularında yardımcı olabilirim" diyerek sohbeti koparmadan kendi iç algoritmasıyla cevap döndürür.	